

DATASCI 350 - Data Science Computing

Danilo Freire

Fall 2026

Contents

Course Description	2
Learning Objectives	2
Course Requirements	2
Materials	3
Course Information	5
Software	5
Office Hours	6
Academic Integrity	6
Artificial Intelligence	7
Special Needs and Accessibility Services	7
English Language Learners	8
Assignments and Grading Policy	8
Grading Scale	9
Course Outline and Suggested Readings	9

Course Description

Welcome to [DATASCI 350](#)! This course introduces key tools in modern data science, focusing on three essential aspects: reliability, reproducibility, and robustness. We will cover the [command-line interface](#), version control with [Git](#) and [GitHub](#), and reproducible reports using [Quarto](#) and [Jupyter Notebooks](#). You will then work with language models, first running one on your own laptop with [Ollama](#), then calling hosted models from Python, and finally building a small retrieval system over your own documents. We will rent machines in the cloud with [Amazon Web Services](#), collect data from web APIs with [requests](#), and process it at scale with [Polars](#), [DuckDB](#), and [Dask](#). We finish with [Docker](#) and containerisation.

By working with real-world datasets and problems, students will gain hands-on experience using these tools and methods to extract insights from data. This course will develop technical skills and critical thinking needed to solve complex data challenges. Upon completion, students will be prepared to apply these tools to their own research and professional work.

Learning Objectives

By the end of this course, students will be able to:

- Use the command line interface to manage files and directories.
- Work with version control systems to track changes in code and collaborate with others.
- Create reproducible reports and presentations.
- Use language models and coding agents responsibly to assist with programming tasks.
- Collect data from web APIs and web pages, and process it at scale.
- Understand the basics of containerisation and parallel computing.

Course Requirements

Some knowledge of programming is recommended, and familiarity with basic data manipulation and visualisation techniques is helpful. However, no prior experience with the tools covered in the course is required.

In terms of software, you will need to install the following tools: [Anaconda distribution of Python 3.x](#), [VS Code](#), [Git](#), [GitHub Desktop](#), [Quarto](#), [Ollama](#), and [Docker](#). Several lectures also need Python packages that Anaconda does not ship. Install them with `pip install ollama openai python-dotenv requests pyarrow polars duckdb dask joblib`. You will open two free accounts during the course, one on [OpenRouter](#) for Lecture 14 and one on [Amazon Web Services](#) for Lectures 16 and 17. Neither costs anything if you follow the steps in the lecture READMEs.

Please feel free to reach out if you have any questions about the course content or your readiness to take the class.

Materials

This course is designed to be self-contained, providing all the necessary resources and materials to succeed in mastering the core concepts. However, students are encouraged to explore the following suggested books, online courses, and documentation to deepen their understanding of the topics covered in the course. Everything listed here is free to read.

Suggested Books

- [Data Science on the Command Line](#) by Jeroen Janssens
- [Pro Git](#) by Scott Chacon and Ben Straub
- [The Turing Way](#) by The Turing Way Community
- [Python for Data Analysis](#) by Wes McKinney
- [Elements of Data Science](#) by Allen Downey
- [Speech and Language Processing](#) by Dan Jurafsky and James H. Martin. Note: the chapters on embeddings and transformers explain what Lectures 12 and 15 use.
- [DuckDB in Action](#) by Mark Needham, Michael Hunger, and Michael Simons
- [Free programming books](#)

Online Courses

- [MIT: The Missing Semester of Your CS Education](#)
- [Coursera: Python for Everybody Specialisation](#)
- [Class Central: Introduction to Git and GitHub](#)
- [Hugging Face: LLM Course](#)
- [AWS Skill Builder: Cloud Practitioner Essentials](#)
- [Kaggle: Learn](#)

Documentation

- [Official Python Documentation](#)
- [NumPy Documentation](#)
- [Pandas Documentation](#)
- [Matplotlib Documentation](#)
- [Quarto Documentation](#)
- [Git Documentation](#)
- [GitHub Documentation](#)
- [Ollama Documentation](#)
- [OpenRouter Documentation](#)
- [Requests Documentation](#)
- [Polars Documentation](#)
- [DuckDB Documentation](#)
- [Dask Documentation](#)
- [AWS Documentation](#)
- [Docker Documentation](#)

Course Information

We will meet every Tuesday and Thursday from 5:30pm to 6:45pm in [Math & Science Center, Room W301](#). It is important that you read the materials before class. All information about the course is available on the course's GitHub repository at <https://github.com/danilofreire/datasci350>. While I will try to adhere to the course schedule as much as possible, I also want to adapt to your learning pace and style. The syllabus and course plan may change in the semester. Again, please check [the course repository](#) regularly to check for updates. I will also announce any changes in class and via email.

Software

We will mainly use [Python](#) in this course. Python is a free and powerful programming language that is widely used in data science, machine learning, and scientific computing. I recommend using the [Anaconda distribution](#) as it comes with many necessary Python libraries for data analysis, such as [Pandas](#), [NumPy](#), and [Jupyter](#).

You can write your Python code in any text editor, but I recommend [VS Code](#) with the [Python extension](#). [Pycharm](#) is also well-regarded by developers. If you are feeling adventurous, you can also use [Neovim](#) with the [coc-pyright](#) plugin. That is, if [you can exit the editor](#). :)

We will collect our own data rather than query a database. Lectures 18 and 19 use [requests](#) to pull data from web APIs, and we save the results as CSV and parquet files. If you want to learn SQL, work through the self-study [DuckDB tutorial](#) in the course repository. DATASCI 151 also covers SQL basics in more depth than we could here.

We will also use [Jupyter Notebooks](#) and [Quarto](#) in class. Jupyter itself comes pre-installed with Anaconda, but please install the [Jupyter extension for VS Code](#) as well. To install Quarto, please follow the instructions on the [official website](#). We will have a hands-on session to learn how to use both of them (but I assume you are already familiar with Jupyter).

Please also install [Docker](#) to work with containers. Docker is a platform for developing, shipping, and running applications in containers. Containers allow you to package your application and its dependencies together into a single unit. This makes it easy to ensure that your application will run on

any other machine, regardless of any custom settings that machine might have that could differ from the machine that was used for writing and testing the code.

Finally, we will use [GitHub](#) for version control. Please create a free account on GitHub and install [GitHub Desktop](#) to manage your repositories. We will also use [Git](#) in the course. Git is a distributed version control system that allows you to track changes in your codebase and collaborate with others. You can install Git from the [official website](#).

To help you get started, I have prepared [six tutorials](#) for the course:

- [Installing VS Code and connecting it with Anaconda](#)
- [Jupyter Notebook and Markdown](#)
- [Opening a free educational account on GitHub](#)
- [SQL essentials with DuckDB](#)
- [Web scraping with Python](#)
- [Setting up WSL on Windows](#)

Read tutorials 01 to 03 in order at the start of the semester. Windows users should also read tutorial 06 before Lecture 03. Tutorials 04 and 05 cover material we do not teach in class, and you can read them whenever you need them.

Office Hours

I am very flexible with office hours, and we can schedule an online meeting at any time that works for you. Feel free to send me a message at daniilo.freire@emory.edu, and I will likely reply within a few hours. If you prefer, you can meet me in the afternoon at my office. My office address is in the [Psychology and Interdisciplinary Sciences Building, 36 Eagle Row, room 480](#). If possible, please email me before coming to ensure that no two students book the same time slot.

Academic Integrity

Upon every individual who is a part of Emory University falls the responsibility for maintaining in the life of Emory a standard of unimpeachable honour in all academic work. The [Honor Code of Emory](#)

[College](#) is based on the fundamental assumption that every loyal person of the University not only will conduct his or her own life according to the dictates of the highest honor, but will also refuse to tolerate in others action which would sully the good name of the institution. Academic misconduct is an offense generally defined as any action or inaction which is offensive to the integrity and honesty of the members of the academic community. Any suspected case of academic misconduct will be referred to the Emory Honor Council.

Artificial Intelligence

Students have to submit ten problem sets and complete five in-class quizzes. You are allowed to use AI to assist with your assignments, and you may also use AI tools during quizzes. However, any errors or omissions resulting from the use of AI tools are your responsibility, and you must always double-check your work. Remember to cite all sources used in your problem sets and projects, including AI tools. Please include a note at the end of any document indicating that AI was used in its development. Quizzes carry an explanation requirement, which is described in the grading policy below. Using AI tools in a manner prohibited in this course syllabus constitutes cheating under the Emory Honor Code and is thus a form of academic misconduct.

Special Needs and Accessibility Services

I am committed to providing necessary accommodations to ensure all students have an equal opportunity to succeed in this course. Students with medical or health conditions that may impact their academic performance should visit the [Department of Accessibility Services \(DAS\)](#) to determine eligibility for appropriate accommodations. Those who receive accommodations should provide me with an Accommodation Letter from DAS at the beginning of the semester or as soon as the accommodation is granted. Please note that DAS accommodations, such as extra time or quiet spaces, will apply only to quizzes, not assignments. This is because assignments are released in advance, allowing students to work at their own pace. Athletes and students with other commitments should also inform me of any scheduling

conflicts at the beginning of the semester. I will do my best to accommodate these students, but I cannot guarantee that all requests will be granted. If you have any questions or concerns, please contact me.

English Language Learners

Emory University welcomes students from around the country and the world, and the unique perspectives international and multilingual students bring enrich the campus community. To empower multilingual learners, an array of support is available including language and culture workshops and individual appointments. For more information about English Language Learning support at Emory, please contact the ELLP Specialists at <https://writingcenter.emory.edu>. No student will be penalised for their command of the English language.

Assignments and Grading Policy

Problem Sets (50%). There will be ten problem sets throughout the course. These assignments are designed to reinforce concepts covered in lectures and readings, and to provide hands-on practice with statistical programming. Problem sets will include a mix of theoretical questions and practical applications. They will be assigned regularly and must be completed individually. All ten problem sets are in the course repository from the first day of class, so you can start any of them whenever you like. The dates in the schedule are the rhythm I mark to, not the moment the work appears. Please acknowledge any sources you used in your work, including textbooks, articles, and AI resources. *Any assignment submitted after the due date/time will be penalised by 10% per day.* Please submit your assignments as Jupyter Notebooks (.ipynb) or .pdf files via Canvas or email until midnight on the due date.

Class Quizzes (30%). Students will also take five in-class quizzes throughout the semester. These quizzes will be based on the lectures from the previous weeks. They will be designed to test your understanding of the material and your ability to apply the concepts to new problems. Quizzes will be open-book and open-notes, and students have the entire class period to complete them. They are individual assessments, and students are not allowed to discuss the questions with their classmates in class. You may use AI tools during a quiz, on the same terms as any other resource. *You must be able to*

explain every command and answer you submit. After each quiz I select about five students at random and ask them to walk me through part of their work. Students who cannot explain what they submitted will lose marks at my discretion, up to and including a zero for the quiz, depending on how much of the work they can account for.

Final Project (20%): For the final project, you will work in groups of three to four to create a short report. This report will require you to apply the tools and methods we have covered in class to a real-world dataset. You should host your report in a GitHub repository, use Quarto for the document, and collect the data through the World Bank API. Make sure to include visualisations and statistical analyses as well. The project is due on the last day of class. You can find the instructions in the course [GitHub repository](#).

Grading Scale

Each student's final grade will be based on the following:

Grade	A	A-	B+	B	B-	C+	C	C-	D+	D	F
Range	93%	90-	87-	83-	80-	77-	73-	70-	67-	60-	<60%
	and	92.9%	89.9%	86.9%	82.9%	79.9%	76.9%	72.9%	69.9%	66.9%	
	above										

Course Outline and Suggested Readings

The lecture notes cover all the necessary material for the course, and the weekly suggested readings are recommended for those who want to deepen their understanding of the course topics. As mentioned above, the course outline is subject to change, and I will update the syllabus if needed. Please remember to check the course [GitHub repository](#) regularly. Lecture notes, assignments, and other materials will be posted there as the course progresses.

Module 01: Introduction

August 27, 2026:

- Syllabus and course repository: <https://github.com/danilofreire/datasci350>.
- Lecture 01: [Welcome to DATASCI 350 - Introduction](#).
- Course tutorials: installing Anaconda and VS Code, Jupyter and Markdown, and opening a free educational account on GitHub.

Suggested references:

- Cleveland, W. S. (2001). [Data science: An action plan for expanding the technical areas of the field of statistics](#). International Statistical Review, 69(1), 21-26.
- Donoho, D. (2017). [50 Years of Data Science](#). Journal of Computational and Graphical Statistics, 26(4), 745-766.
- Breiman, L. (2001). [Statistical Modeling: The Two Cultures \(with Comments and a Rejoinder by the Author\)](#). Statistical Science, 16(3), 199-231.
- Brady, H. E. (2019). [The Challenge of Big Data and Data Science](#). Annual Review of Political Science, 22(1), 297-323.

September 1, 2026:

- Lecture 02: [Computational Literacy](#).

Suggested references:

- Campbell-Kelly, M., Aspray, W. F., Yost, J. R., Tinn, H., & Díaz, G. C. (2023). [Computer: A History of the Information Machine](#). Routledge.
- Shalf, J. (2020). [The Future of Computing beyond Moore's Law](#). Philosophical Transactions of the Royal Society A, 378(2166), 20190061.
- Wing, J. M. (2006). [Computational Thinking](#). Communications of the ACM, 49(3), 33-35.
- Videos: [David J. Malan - Abstraction](#), [Khan Academy - Hexadecimal Number System](#), [Matthias Wandel - Marble Adding Machine](#), [Crash Course - Early Computing and Electronic Computing](#) (the last two are quite entertaining!).

Module 02: Introduction to the Command-Line Interface and Version Control

September 3, 2026:

- Lecture 03: [The Command-Line Interface \(CLI\), Shell Basics, and File Management.](#)
- **Assignment 01.**
- [Kahoot Quiz](#)

Suggested references:

- Janssens, J. (2021). [Data Science at the Command Line: Obtain, Scrub, Explore, and Model Data with Unix Power Tools](#) (2nd ed.). O'Reilly Media.
- Levy, J. (2024). [The Art of Command Line](#). GitHub.
- Shotts, W. (2019). [The Linux Command Line: A Complete Introduction](#). No Starch Press.
- Healy, K. (2019). [The Plain Person's Guide to Plain Text Social Science](#). Chapters 1-5.

September 8, 2026:

- Lecture 04: [More Command Line Tools, Text Files and Scripting.](#)
- [Kahoot Quiz.](#)

Suggested references:

- Kerr, D. (2024). [Effective Shell.](#)
- Irianto, I. (2021). [Learn Vim \(the Smart Way\).](#)
- Dennis, J. [Your problem with Vim is that you don't grok vi.](#) (Stack Overflow).
- Videos: [freeCodeCamp - Command line crash course](#), [Percy Grunwald - Absolute beginner guide to the macOS terminal](#), [NetworkChuck - 50 macOS tips and tricks using terminal](#)

September 10, 2026:

- Lecture 05: [Version control with Git and GitHub.](#)
- **Assignment 01 due (5%).**
- **Assignment 02.**

- [Kahoot Quiz](#).

Suggested references:

- Chacon, S. and Straub, B. (2014). [Pro Git](#). Apress. (Instructor's note: this is *the book* on Git).
- GitHub tutorials: [GitHub skills](#) (recommended), [Git guides](#), [Best practices for repositories](#).
- [Dangit, Git!?!.](#) Note: the fix for the commit you did not mean to make, in plain language.

September 15, 2026:

- Lecture 06: [More Git and GitHub: pull requests, issues, pages, and collaboration features](#).
- [Kahoot Quiz](#).

Suggested references:

- Perez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Leprevost, F. da V., Fufezan, C., Ternent, T., Eglen, S. J., Katz, D. S., Pollard, T. J., Konovalov, A., Flight, R. M., Blin, K., & Vizcaíno, J. A. (2016). [Ten Simple Rules for Taking Advantage of Git and GitHub](#). PLOS Computational Biology, 12(7), e1004947.
- Beckman, M. D., Çetinkaya-Rundel, M., Horton, N. J., Rundel, C. W., Sullivan, A. J., & Tackett, M. (2021). [Implementing version control with git and GitHub as a learning objective in statistics and data science courses](#). Journal of Statistics and Data Science Education, 29(sup1), S132-S144.
- Escamilla, E., Klein, M., Cooper, T., Rampin, V., Weigle, M. C., & Nelson, M. L. (2022). [The Rise of GitHub in Scholarly Publications](#). arXiv preprint arXiv:2208.04895.

September 17, 2026:

- Lecture 07: [More Git Commands, CLI and Git Practice](#).
- **Assignment 02 due (5%).**
- **Assignment 03.**
- [Kahoot Quiz](#).

September 22, 2026:

- Lecture 08: [Review session for Quiz 01](#). This session revises everything covered so far and prepares students for Quiz 01 on September 24, which is examinable on Lectures 02-07.

Module 03: Reproducible Research with Quarto

September 24, 2026:

- Lecture 09: [Quiz 01: The Command Line, Git and GitHub \(6%\)](#). Covers Lectures 02-07.
- **Assignment 03 due (5%).**
- **Assignment 04.**

Suggested references:

- [Markdown Guide](#).
- [Jupyter Notebooks Documentation](#).
- [Course tutorial: Jupyter and Markdown](#)

September 29, 2026:

- Lecture 10: [Reproducible Research and Literate Programming](#).

Suggested references:

- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). [Ten Simple Rules for Reproducible Computational Research](#). PLOS Computational Biology, 9(10), e1003285.
- [The Turing Way: Guide for Reproducible Research](#).
- [Quarto: Get Started](#).
- Çetinkaya-Rundel, M. & Lowndes, J. S. (2022) [Keynote talk: Hello Quarto: Share • Collaborate • Teach • Reimagine](#). [Slides](#) and [source code](#). This is one of the nicest Quarto presentations I have seen.

October 1, 2026:

- Lecture 11: [Quarto in Practice](#).
- **Assignment 04 due (5%).**
- **Assignment 05.**
- [Kahoot Quiz](#)

Suggested references:

- [Quarto Documentation - Presentations and Websites](#).
- [Quarto Documentation - Code Execution and Freeze](#).
- [GitHub Pages Documentation](#).
- French, J. (2023). [Creating Websites with Quarto and GitHub Pages \(YouTube Playlist\)](#).

Module 04: AI-Assisted Programming

October 6, 2026:

- Lecture 12: [Local Language Models](#).
- [Kahoot Quiz](#).

Suggested references:

- [Ollama Documentation](#) and the [Modelfile Reference](#).
- [Ollama Model Library](#). Note: browse this before class and pick a model your laptop can hold.
- Wolfram, S. (2023). [What Is ChatGPT Doing, and Why Does It Work?](#).
- Google (2025). [Gemini Prompting Guide](#). Note: the source of the PTCF framework we use to write system prompts.

October 8, 2026:

- Lecture 13: [Quiz 02: Reproducible Research and Quarto \(6%\)](#). Covers Lectures 10 and 11.
- **Assignment 05 due (5%).**
- **Assignment 06.**

October 13, 2026: Fall Break (No Classes)

October 15, 2026:

- Lecture 14: [Calling Models from Your Own Code](#).

Note: create a free [OpenRouter](#) account and API key before this class, and set the key's spending limit to \$0.00. The steps are in the lecture README.

Suggested references:

- [Ollama API Documentation](#) and its [OpenAI compatibility](#) page. Note: lecture 12 runs Ollama from the terminal, and this lecture calls the same server from Python.
- [OpenRouter Quickstart](#) and the [model catalogue](#). Note: filter the catalogue by Prompt pricing "Free" to find a model that costs nothing today.
- [OpenAI Python library](#). Note: despite the name, it is a client for a protocol that Ollama and OpenRouter both speak.
- Willison, S. (2025). [The lethal trifecta for AI agents](#).

October 20, 2026:

- Lecture 15: [Retrieval-Augmented Generation and Fine-Tuning](#).

Note: install or update Ollama before this class, then run `pip install ollama numpy`. The steps are in the lecture README. - [Kahoot Quiz](#).

Suggested references:

- [Ollama Documentation: Embedding models](#).
- [EmbeddingGemma on the Ollama model library](#). Note: this is the model we run in class.
- [Sentence Transformers Documentation](#). Note: the clearest explanation of what a sentence embedding is and how similarity search uses it.
- [Unsloth Documentation](#). Note: the tool behind the fine-tuning demonstration; you are not expected to run it yourself.

Module 05: Cloud Computing

October 22, 2026:

- Lecture 16: [Introduction to Cloud Computing](#).
- **Assignment 06 due (5%).**
- **Assignment 07.**
- [Kahoot Quiz](#).

Suggested references:

- [Amazon Web Services \(AWS\) Documentation](#).
- [AWS Free Tier](#). Note: read this before opening an account, and set a zero-spend budget.
- [AWS Cloud Practitioner Tutorial](#).

October 27, 2026:

- Lecture 17: [Cloud Computing II: Running Your Own Server on AWS](#).
- No new assignment this week. Start reading the final project instructions.
- [Instructions for the Final Project](#).
- [Kahoot Quiz](#).

Suggested references:

- [Amazon EC2 Documentation](#).
- [AWS CloudShell Documentation](#). Note: the way out when a key file's permissions will not work on Windows.
- [Ubuntu Server Documentation](#). Note: the instance we launch runs Ubuntu, and apt is how you install anything on it.

Module 06: Getting Data from the Web

October 29, 2026:

- Lecture 18: [Web APIs and JSON](#).

- **Assignment 07 due (5%).**
- [Kahoot Quiz](#).

Suggested references:

- [Requests: Quickstart](#).
- MDN. [An overview of HTTP](#).
- [World Bank API Documentation](#). Note: the API behind the final project.
- [Working with JSON](#).

November 3, 2026:

- Lecture 19: [Working with APIs in Practice](#).
- Optional: [Web scraping tutorial](#). Self-study, not examinable.
- [Kahoot Quiz](#).

Suggested references:

- [pandas: json_normalize](#). Note: how nested JSON becomes a flat table.
- [python-dotenv Documentation](#). Note: how an API key stays out of your repository.
- [Apache Parquet Documentation](#). Note: the format we save results in, and the one Lectures 21 and 22 read back.
- [wbgapi](#). Note: a Python client for the World Bank API, useful for the final project.

November 5, 2026:

- Lecture 20: [Quiz 03: AI-Assisted Programming and Cloud Computing \(6%\)](#). Covers Lectures 12, 14, 15, 16, and 17.
- **Assignment 08.**

Module 07: Scaling and Performance

November 10, 2026:

- Lecture 21: [Parallel Computing Fundamentals](#).

Suggested references:

- [Python Documentation: concurrent.futures](#).
- [Python Glossary: global interpreter lock](#).
- [joblib Documentation](#).
- [Dask Documentation](#).

November 12, 2026:

- Lecture 22: [Scaling Data Analysis in Practice](#).
- **Assignment 08 due (5%).**
- **Assignment 09.**
- [Kahoot Quiz](#).

Suggested references:

- [Polars User Guide](#).
- [DuckDB Documentation](#).
- [Dask: Best Practices](#).

Module 08: Containers and Reproducibility

November 17, 2026:

- Lecture 23: [Dependency Management, Virtual Environments, and Containers](#).
- [Kahoot Quiz](#).

Suggested references:

- [Python Documentation: venv](#).
- [uv Documentation](#). Note: the tool we use in class to create environments and lock dependencies.
- [Docker: Get Started](#).

November 19, 2026:

- Lecture 24: [Quiz 04: Web APIs and JSON \(6%\)](#). Covers Lectures 18 and 19.
- **Assignment 09 due (5%).**
- **Assignment 10.**

November 24, 2026:

- Lecture 25: [Docker for Data Science](#).
- [Kahoot Quiz](#).

Suggested references:

- [Dockerfile Reference](#).
- [Docker Compose Documentation](#).
- [Jupyter Docker Stacks](#). Note: ready-made images for data science work.

November 26, 2026: Thanksgiving Recess (No Classes)

December 1, 2026:

- Lecture 26: [Course Revision](#).
- [Kahoot Quiz](#).

December 3, 2026:

- Lecture 27: [Quiz 05: Scaling, Docker and Containers \(6%\)](#). Covers Lectures 21, 22, 23, and 25.
- **Assignment 10 due (5%).**

December 8, 2026:

- No lectures.
- Q&A session and final project work time.
- [Final Project due \(20%\)](#).